Shuttle Dispatch

Value and Policy Iteration

The infinite-horizon solution is to **wait when fewer than 13 passengers are waiting and dispatch when 13 or more are waiting**. The sections below specify the model, explain each solution method, and present numerical evidence that the implementation is correct.

1 The model: what happens in one period?

Each period has four steps: observe the queue, choose whether to dispatch, charge the dispatch and waiting costs, and then add the new arrivals. The state records only the queue length because the assumed arrival distribution and available actions do not otherwise depend on the past.

Table 1: Notation used to describe the environment.

Symbol	Meaning
t	Decision period; the first decision is at $t = 0$.
s_t	Number of passengers waiting *before* the decision at time t. Possible states are $0, 1, \ldots, 200$.
a_t	Action: 0 means wait; 1 means dispatch one shuttle.
N_t	Random number of new arrivals after the decision. Each of $1, 2, 3, 4, 5$ has probability $1/5$. Arrivals are independent across periods.
$c(s, a)$	Cost of taking action a when the queue contains s passengers.
γ	Discount factor, 0.95. A cost one period later receives weight 0.95 relative to a cost now.

A shuttle can serve 15 passengers. Therefore, after action a in state s, the number still waiting is $\max\{s - 15a, 0\}$. The maximum with zero prevents the queue from becoming negative. The cost is

$$\underbrace{c(s,a)}_{\text{current cost}} = \underbrace{100a}_{\text{dispatch cost}} + \underbrace{2\max\{s-15a,0\}}_{\text{cost of passengers left waiting}} . \tag{1}$$

New arrivals are added after this cost is assessed. The next queue is

$$s_{t+1} = \min\{\max\{s_t - 15a_t, 0\} + N_t,\ 200\}\,. \tag{2}$$

The minimum with 200 enforces the station capacity.

Example. With 20 passengers, waiting costs $2(20) = 40$ and leaves 20 passengers before arrivals. Dispatching serves 15, leaves 5, and costs $100 + 2(5) = 110$. If three passengers then arrive, the next queue is 23 after waiting or 8 after dispatch. Dispatching costs more *now*, but may save enough future waiting cost to be optimal.

Additional assumptions. Only one shuttle can depart per period, with no separate fleet or travel-time restriction. New arrivals first incur waiting cost at the next decision. Overflow above 200 is discarded without an extra penalty. These timing and overflow choices are assumptions, rather than consequences of the queue cap alone. Ties between actions are resolved by waiting.

2 Finite-horizon enumeration: 500 decisions

2.1 The quantity being optimized

Let $J_t(s)$ be the **minimum expected discounted cost still to be paid**, starting with s passengers at time t. Costs are discounted relative to that time. With 500 decisions, its definition is

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right]. \tag{3}$$

Here u counts future periods; π denotes a policy, meaning a rule for choosing actions; and $\mathbb{E}_{\pi}$ means an average over arrival randomness when that policy is followed. A smaller $J_t(s)$ is better. The report uses *cost values*; if rewards were defined as negative costs, their values would have the opposite sign.

2.2 Backward induction over states and actions

Part (a)'s "enumeration" is interpreted as evaluating every state, both actions, and all five possible arrivals at each period. There is no terminal penalty, so $J_{500}(s) = 0$. Starting at the last decision and working backward, compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5}\sum_{k=1}^{5} J_{t+1}(\min\{\max\{s-15a, 0\} + k, 200\}), \tag{4}$$

$$J_t(s) = \min_{a\in\{0,1\}} Q_t(s,a). \tag{5}$$

$Q_t(s,a)$ is the expected remaining cost if action a is taken *now* and decisions are optimal afterward. In the sum, k is a possible arrival count. The factor $1/5$ averages the five outcomes, and γ discounts their continuation costs. The minimizing action is saved as $\pi_t(s)$, the decision rule at time t.

This procedure reuses previously computed continuation values, avoiding the exponential cost of listing every complete decision history. The implementation saves all 501 value vectors and all 500 policy vectors.

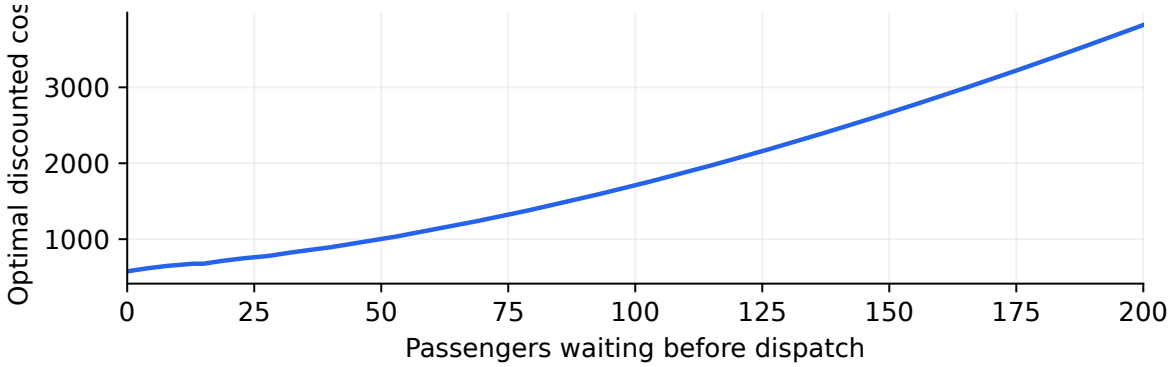


Figure 1: Minimum expected discounted cost at the initial decision, $J_0(s)$, for 500 decisions and zero terminal cost.

The finite-horizon policy need not be stationary: it can change as the end approaches. In the final period, waiting is optimal in every state. A dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately; there is no later benefit or terminal penalty to justify it.

3 Infinite-horizon value iteration

For an unlimited sequence of decisions, write $J^*(s)$ for the minimum expected discounted cost starting from queue s. The superscript $*$ indicates optimality. This value has no time subscript because the problem has no deadline and the environment does not change over time.

3.1 The Bellman update

Starting from $J^{(0)}(s) = 0$, repeatedly compute

$$J^{(n+1)}(s) = \min_{a\in\{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{\max\{s-15a,0\}+k, 200\}) \right]. \tag{6}$$

The superscript (n) counts algorithm iterations, not decision periods. Each update asks: *which action has the lowest current cost plus discounted expected future cost?* Its structure is the same as the finite-horizon recursion, but it improves an estimate of the stationary value instead of moving backward through time.

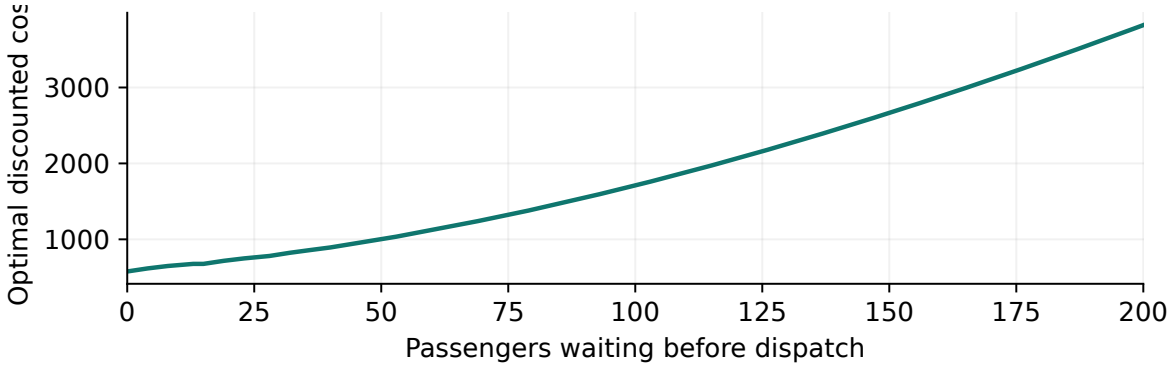


Figure 2: Optimal infinite-horizon cost $J^*(s)$, estimated by value iteration. Larger initial queues incur greater total expected cost.

3.2 When does iteration stop?

Let $\mathcal{T}J$ denote the entire right-hand side of Equation (6), evaluated using a candidate value vector J. This is the *Bellman operator*: one application performs one value update. Define its residual as

$$\delta(J) = \max_{s\in\{0,\ldots,200\}} |(\mathcal{T}J)(s) - J(s)|. \tag{7}$$

The residual measures how much another Bellman update would change the candidate values, in the worst state. The algorithm stops when $\delta(J) \le 10^{-10}$, which required **518 updates**.

Because $\gamma < 1$, the Bellman operator contracts maximum errors by at most γ. Consequently,

$$\max_s |J(s) - J^*(s)| \le \frac{\delta(J)}{1-\gamma}. \tag{8}$$

This connects a computable stopping criterion to the accuracy of the value estimate.

The implementation minimizes costs. Thus, its arrays named `finite_costs`, `vi_cost`, and `pi_cost` are the cost values J_t and J^* described above.

4 Policy iteration and the dispatch rule

Policy iteration alternates between evaluating a fixed decision rule and improving it. It starts from the rule "always wait."

Policy evaluation. For a stationary policy π, $\pi(s)$ is its action in state s. Let J_π be the vector of expected discounted costs when that rule is followed indefinitely. Let P_π be the 201×201 transition matrix: entry (s, s') is the probability of next queue s' after taking action $\pi(s)$. Let c_π be the vector with entries $c(s, \pi(s))$. Then

$$J_\pi = c_\pi + \gamma P_\pi J_\pi, \qquad \text{so} \qquad (I - \gamma P_\pi) J_\pi = c_\pi. \tag{9}$$

Here I is the identity matrix, and s' denotes a possible next state. The linear system computes the cost of the current policy. When several arrival counts produce the same next state, their probabilities are added in P_π.

Policy improvement. For every state, compare both actions using J_π as the continuation value in Equation (6). Replace the current action with the lower-cost one. Stop when no actions change. This took **six evaluation/improvement rounds**, including the final stability check.

The resulting optimal stationary rule is

$$\boxed{\pi^*(s) = \begin{cases} 0 & \text{(wait)}, \quad 0 \le s \le 12, \\ 1 & \text{(dispatch)}, \quad 13 \le s \le 200. \end{cases}} \tag{10}$$

The threshold was discovered by comparing both actions in every state; it was not imposed on the search. Departing with 13 or 14 passengers can be worthwhile because the savings in future holding costs outweigh the benefit of waiting for a full shuttle.

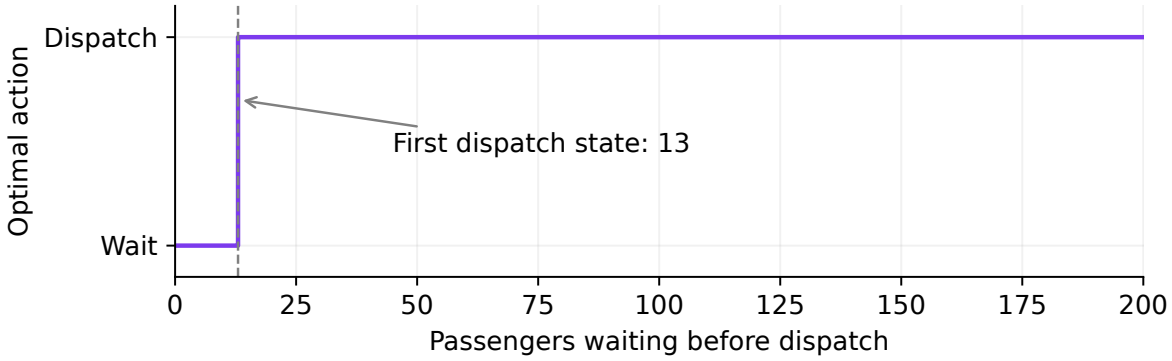


Figure 3: Optimal stationary policy from policy iteration. The first dispatch state is 13.

Table 2: Representative minimum infinite-horizon discounted costs.

Initial passengers s	Optimal cost $J^*(s)$
0	576.670168
15	676.670168
200	3821.586477

The difference $J^*(15) - J^*(0) = 100$ is a useful consistency check. Dispatch at 15 clears the queue, while waiting at zero also leaves it empty. Both lead to the same next-arrival distribution; only the dispatch charge differs.

5 Edge cases and correctness checks

All **14 validation checks passed**. The checks use optimality conditions, independent solution procedures, and model changes with predictable outcomes.

5.1 Agreement and optimality

Value iteration and policy iteration selected identical actions in all 201 states. Their maximum value difference was 1.964×10^{-9}. The policy-iteration result had Bellman residual 9.095×10^{-13}. Equation (8) therefore bounds its maximum value error by approximately 1.819×10^{-11}, subject to floating-point rounding. The smallest absolute difference between the two action costs was 3.778, well above the numerical error.

The maximum difference between the 500-period time-zero values and infinite-horizon values was 4.942×10^{-9}. Since the largest possible stage cost is 470, a conservative bound on the discounted cost beyond 500 periods is

$$\sum_{t=500}^{\infty} \gamma^t \, 470 = \frac{470\gamma^{500}}{1-\gamma} = 6.838 \times 10^{-8}. \tag{11}$$

The observed difference is smaller than this bound. Agreement is expected because very late costs receive little weight under discounting.

5.2 Queue boundaries and the final period

Service never produces a negative queue: 15 passengers clear exactly, and 16 leave one. At state 200, waiting always gives next state 200; dispatch gives next states 186 through 190. At zero, dispatch costs 100 more than waiting without changing the next-state distribution, so it is strictly dominated. These cases verify that service, saturation, and probability pooling are implemented consistently.

The terminal value vector was checked to be zero. The last-period policy was also checked against the analytic prediction that waiting is optimal everywhere. This tests that the finite-horizon algorithm does not charge continuation costs beyond its deadline.

Table 3: Model modifications with known answers. In this table, $J(s)$ denotes the optimal cost for the modified problem.

Modification	Predicted result	Observed result
Zero holding cost	Always wait; $J(s) = 0$.	Matched all states.
Zero shuttle capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$.	Zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty states; $J(s) = 0$.	Matched all states.
Zero discount factor	Only current costs matter: always wait, with $J(s) = 2s$.	Matched all states.

A separately coded exhaustive decision/arrival tree for a seven-state, four-period example matched backward induction exactly. Complete enumeration is feasible at this scale and provides an independent check of the finite-horizon recursion.

6 Independent simulation and reproducibility

The optimal policy was simulated from initial queues of 0, 15 and 200. Each starting state used 20,000 independent trajectories of 300 decisions, with random seed 1668. A trajectory's total cost is the sum of its stage costs weighted by $1, \gamma, \gamma^2, \ldots$. The simulator calculated transitions and costs directly instead of reusing the solver's transition-array builder.

Table 4: Simulation estimates compared with exact policy evaluation.

Initial queue	Simulation mean	Exact cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

The standard error estimates uncertainty in the simulation *mean*: it is the sample standard deviation of trajectory costs divided by $\sqrt{20{,}000}$. Each mean was within four standard errors of the exact value, allowing an additional 0.001951 for the conservative bound on costs omitted after period 300. This wider tolerance is the automated simulation check; Figure 4 displays approximate 95% intervals using ± 1.96 standard errors.

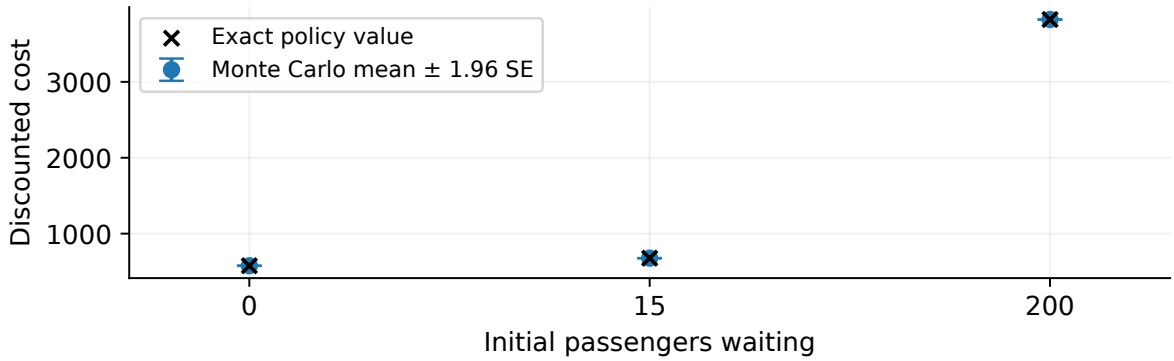


Figure 4: Independent simulation means and exact policy costs. Error bars describe uncertainty in the mean, not the variation of individual trajectory costs.

Simulation supports the computed policy values but does not prove optimality. The Bellman residual provides the stronger optimality check. Likewise, algorithm agreement alone cannot catch every modeling mistake because the methods share the same specification; the analytic examples and independently coded tree and simulator provide additional evidence.

6.1 Reproducing the results

From the project folder, run:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and all validation checks, raising an error if any check fails. Full evidence is saved in `results/summary.json`; per-state results are in `results/state_results.csv`. The saved arrays in `results/shuttle_results.npz` include every finite-horizon value and policy. Running all notebook cells regenerates the assignment plots in `results/figures/`. For the compact vector figures used in this report, run `make_report_figures.py`; then compile `SHUTTLE_REPORT.tex`.

7 Validation with an additional three months

An extended study would focus on three questions.

How sensitive is the dispatch rule to the parameters?

Vary dispatch and holding costs, shuttle capacity, discounting, and arrival distributions. Record changes in the optimal threshold and test whether threshold behavior persists. Compare larger queue caps, such as 250, 500 and 1,000, and introduce penalties for lost arrivals. These experiments would reveal whether the 13-passenger recommendation depends strongly on the chosen parameters or overflow assumption.

Can another method confirm the solution?

Construct an independent linear-programming formulation and compare its values and actions with dynamic programming. Use higher precision when action costs are nearly tied. Vary the finite horizon and terminal penalties to study decisions near the deadline. Use more simulation seeds and starting queues, and measure how often nominal confidence intervals contain the exact policy value.

Does the policy remain effective under more realistic conditions?

Replace independent uniform demand with bursty or correlated arrivals, and evaluate the original policy under these alternative demand processes. Add shuttle availability and travel times to the state. These extensions would test robustness to modeling assumptions, beyond verifying numerical correctness for the stated environment.

Policy Gradient: CartPole and Pong

Assignment 1 — Parts 1 and 2

October 26, 2026

GitHub username: FailedMath3U
Repository name: MLDS
Repository link: `https://github.com/FailedMath3U/MLDS`

Scope and main findings

This report covers the policy-gradient portions of the assignment: basic policy gradient for CartPole-v1 and Pong-v5, followed by both environments with a baseline. The shuttle-dispatch section is a separate report.

The primary CartPole model without a baseline achieved 500 in every evaluation episode. CartPole's learned baseline reduced gradient variance at a frozen initial policy, but its primary training run performed worse. Pong improved modestly over an untrained policy, but **neither Pong method learned to win**: all 500 evaluation games for each final model were losses. The report preserves these unsuccessful outcomes rather than treating completion of training as successful task learning.

Table 1: Primary final-checkpoint results. Each row uses 500 fresh evaluation episodes with stochastic actions. SD is the population standard deviation of those episode rewards. CartPole uses the preselected training seed 1668; Pong has one training seed.

Environment	Method	Mean reward	SD
CartPole-v1	Basic policy gradient	500.000	0.000
CartPole-v1	With learned baseline	260.308	57.013
Pong-v5	Basic policy gradient	-18.610	1.681
Pong-v5	With constant EMA baseline	-18.446	1.851

Guide to the required material

Assignment requirement	Location in this report
Models, discounted objective, Adam and baseline choices	Sections 1–3
Part 1(a): CartPole curves, histogram, mean and SD	Section 4
Part 1(b): Pong curves, histogram, mean and SD	Section 5
Part 2: repeat both experiments with baselines	Sections 6–7
Edge cases and quantitative correctness evidence	Sections 9–12
Execution and validation instructions	Section 13
Additional three months of experiments	Section 14

1 Environment setup and basic policy gradient

The implementations use PyTorch and REINFORCE-style Monte Carlo policy gradients [1]. Gymnasium is used as the maintained successor to Gym. The exact environment IDs are `CartPole-v1` and `ALE/Pong-v5`; the latter is the official namespaced Pong-v5 environment [2, 3]. No v0 or v4 substitute is used.

1.1 What the symbols mean

Symbol	Meaning
t, T	Decision index within a complete episode, and that episode's length.
s_t	Policy input before decision t: four physical measurements for CartPole, or image-derived features for Pong. It denotes an observation, not a claim of full observability.
a_t, r_t	Action selected at t and reward received immediately afterward.
θ, $\pi_\theta(a \mid s)$	Policy-network parameters and the probability of action a at input s.
γ, G_t	Discount factor and discounted reward remaining from decision t.
b_t	Baseline subtracted from G_t; zero for Part 1.
B, i	Number of complete episodes in an update, and the index of one episode.

The objective is the expected discounted reward of a complete episode:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k. \tag{1}$$

The expectation averages environment and action randomness. G_t is computed backward using $G_t = r_t + \gamma G_{t+1}$, with zero continuation after the episode. The plots instead show **undiscounted episode reward**, $\sum_t r_t$, as required by the assignment.

For each complete batch, Adam minimizes

$$\mathcal{L} = -\frac{c}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})(G_t^{(i)} - b_t^{(i)}). \tag{2}$$

Here c is a fixed positive scale: 0.05 for CartPole and 1 for Pong. Scaling changes gradient magnitude but not the objective's optimum. The outer γ^t is retained for the stated discounted episodic objective. In Part 1, $b_t = 0$: returns are not centered or standardized, and no implicit baseline is introduced.

A logit is an unnormalized action score; softmax converts the two scores into probabilities. Actions are sampled, including during evaluation. Gradients accumulate over the complete batch before an update; Pong computes them in memory-saving chunks without stepping between chunks.

2 Baselines, task details, and evaluation protocol

2.1 CartPole: learned state-and-time baseline

CartPole observes cart position, cart velocity, pole angle and pole angular velocity. Actions push left or right. Default rewards are +1 per transition, including failure; an episode ends on physical failure or truncation at 500 decisions [2]. Both flags finish the return calculation, and accumulation never crosses an episode boundary.

The Part 2 baseline $b_\psi(s_t, t)$ is a separate network: five inputs (the four observations plus $t/500$), 64 tanh units and one output predicting G_t. ψ denotes its weights. Predictions use weights from earlier batches and are detached from the actor loss. After the actor update, five Adam steps at learning rate 0.01 minimize mean squared error between predictions and full Monte Carlo targets over the batch's observed decisions. There is no temporal-difference bootstrapping.

2.2 Pong: visual input and constant baseline

Pong retains v5 defaults: frame skip 4, sticky-action probability 0.25, and an emulator limit of 108,000 frames. Discounting is per agent decision, not per raw emulator frame. The policy samples **only ALE actions 2 and 3** (RIGHT and LEFT), as assigned; no FIRE or NOOP startup action is added.

The RGB image is cropped to rows 35–194, downsampled by two, and its red channel is binarized after removing background colors 144 and 109, matching the provided preprocessing. The policy receives the flattened difference between consecutive 80×80 binary frames, as float32 at inference. The input is zero on reset. This supplies motion information, but is not guaranteed to be a sufficient Markov state. Full games, not scoring rallies, are episodes; reward-to-go is *not* reset after a point.

For Part 2, the baseline is a scalar constant within a batch. Starting at $b_0 = 0$, update it only after using the current batch:

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k. \tag{3}$$

The index k counts updates, not game decisions. Thus the current batch uses only prior data to set its baseline.

An action-independent baseline does not change the expected *unclipped* policy gradient: conditional on the current input and history,

$$\mathbb{E}_a[b \nabla_\theta \log \pi_\theta(a \mid s)] = b \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0. \tag{4}$$

Here ∇_θ is the vector of derivatives with respect to policy parameters, and $\mathbb{E}_a$ averages over sampled actions. Subtracting a predictable return can remove variation unrelated to which action was chosen; it does not change game rewards.

3 Training configuration and evaluation

Table 2: Actor settings. Both methods within an environment share these settings and start from identical actor parameters.

Setting	CartPole	Pong
Discount factor	0.95	0.99
Hidden layer	64 tanh units	200 ReLU units
Output	2 categorical logits	2 categorical logits
Adam learning rate	0.01	0.001
Episodes per update	32	8
Training episodes per method/seed	8,000	20,000
Training seeds	1668, 1669, 1670	1668
Gradient-norm cap	10	5
Resource	CPU, no GPU	One GPU only

3.1 Checkpoint and evaluation choices

The main results use **final checkpoints after fixed budgets**; every model receives 500 fresh stochastic evaluation episodes, with no learning. Reset seeds are separate from training and shared across methods. CartPole's primary seed 1668 was fixed before full experiments; all three seeds are retained. Pong also saved validation-selected checkpoints using 32 reserved games every 1,024 training games; their separate test results are supplementary, not substituted for final-checkpoint results.

Pong uses the permitted alternative learning rate 0.001. On identical real-game data, one update at 0.01 changed the policy more (mean KL divergence 3.423×10^{-4}) than 0.001 (1.996×10^{-6}). KL divergence measures change in the action distribution. This supports a conservative step-size choice, *not* a claim that 0.01 was proven unstable. All CartPole gradients stayed below their cap. Pong clipped 1,542/2,500 basic-method updates (61.68%) and 466/2,500 baseline updates (18.64%). Clipping is an additional stabilizing modification; the formal unbiasedness argument does not extend to the clipped estimator.

3.2 Resource and software record

CartPole used CPU only. Pong used one RTX 5060 Ti (GPU 0), with both actors in one GPU process and eight CPU emulator workers. It completed both 20,000-game training runs and all test evaluations in approximately 2.64 hours, below the 16-hour training ceiling. Only one training seed was used for Pong; there are no unreported Pong training-seed replicates.

Both environments used Python 3.12, PyTorch 2.14.0+cu130, NumPy 2.5.3 and Gymnasium 1.2.2; Pong additionally used ALE 0.11.2. The CUDA-enabled PyTorch installation was reused for CartPole, but CartPole models and tensors stayed on CPU. Exact configurations and source hashes are saved with the results.

4 Part 1(a): CartPole without a baseline

The primary final policy scored **500.000 ± 0.000** over 500 evaluation episodes. All 500 reached the time limit. This establishes strong performance for this model under the evaluated reset distribution, not universal convergence across seeds.

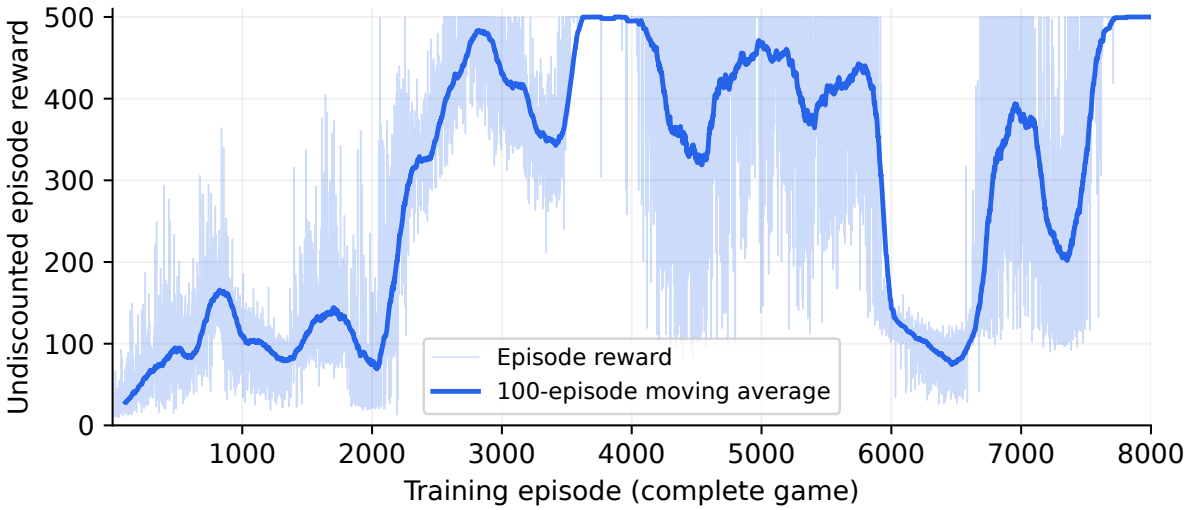


Figure 1: CartPole, Part 1: every training episode reward with the trailing 100-episode moving average. The average begins at episode 100. Seed 1668; 8,000 episodes.

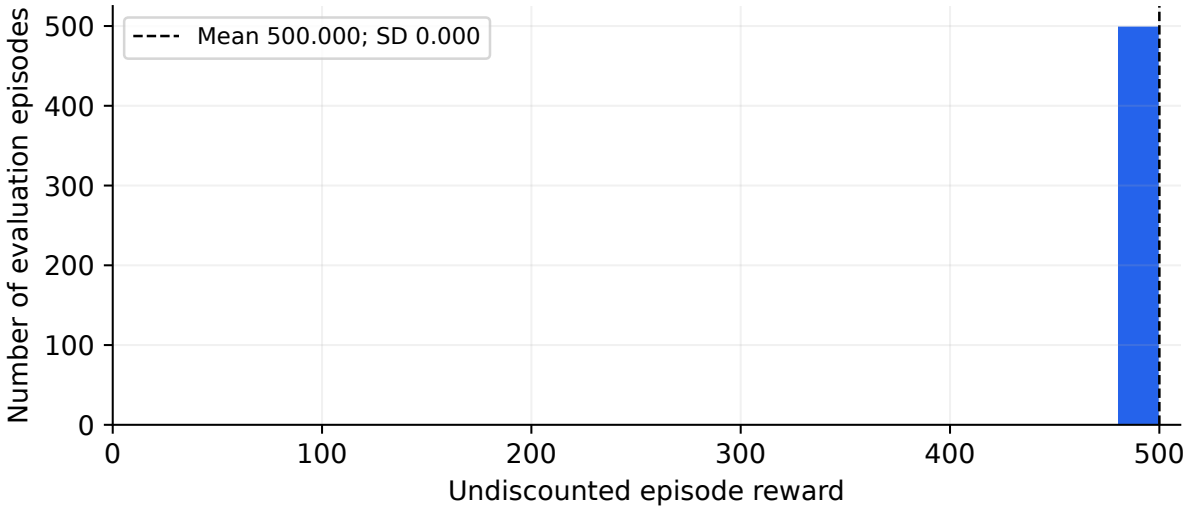


Figure 2: CartPole, Part 1: histogram of all 500 final-policy evaluation rewards. Every reward is 500; all mass is in the last bin. Mean 500.000, population SD 0.000.

The untrained primary neural policy scored 20.926±11.467 over 500 episodes. A uniform random policy scored 22.782±12.058 over another 500 episodes. These references demonstrate substantial improvement beyond initialization or random action selection. The training curve also shows instability before the final plateau, so a favorable final result alone should not be interpreted as uniformly stable learning.

5 Part 1(b): Pong without a baseline

After 20,000 full games, the final policy scored **-18.610 ± 1.681** over 500 evaluation games. Scores ranged from −21 to −10; all games were losses and ended normally, not at the time limit. The task goal of learning to win was **not achieved**.

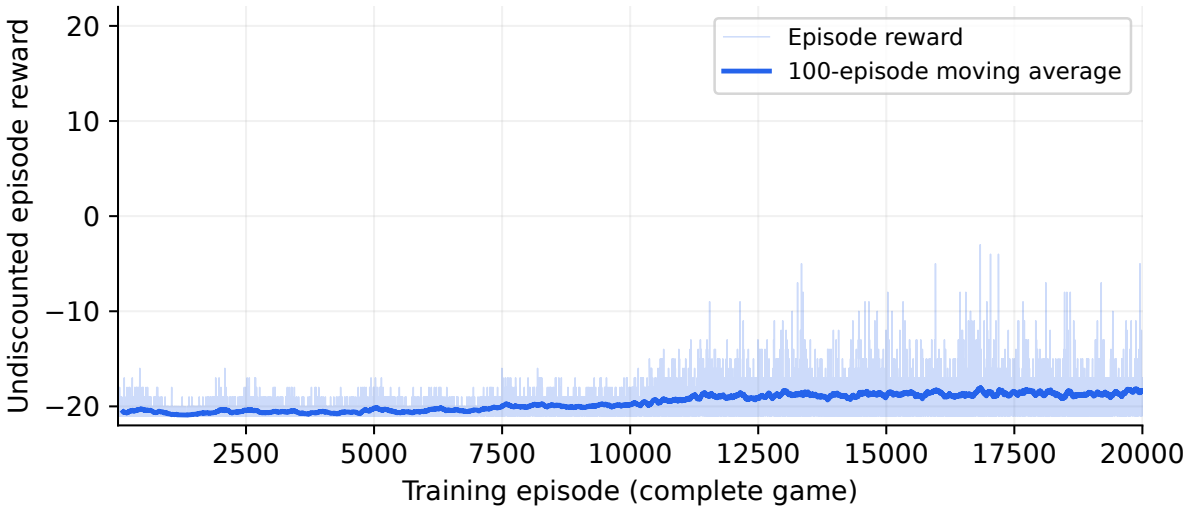


Figure 3: Pong, Part 1: full-game rewards and their trailing 100-game moving average. No rally or point is counted as an episode. Seed 1668; 20,000 games.

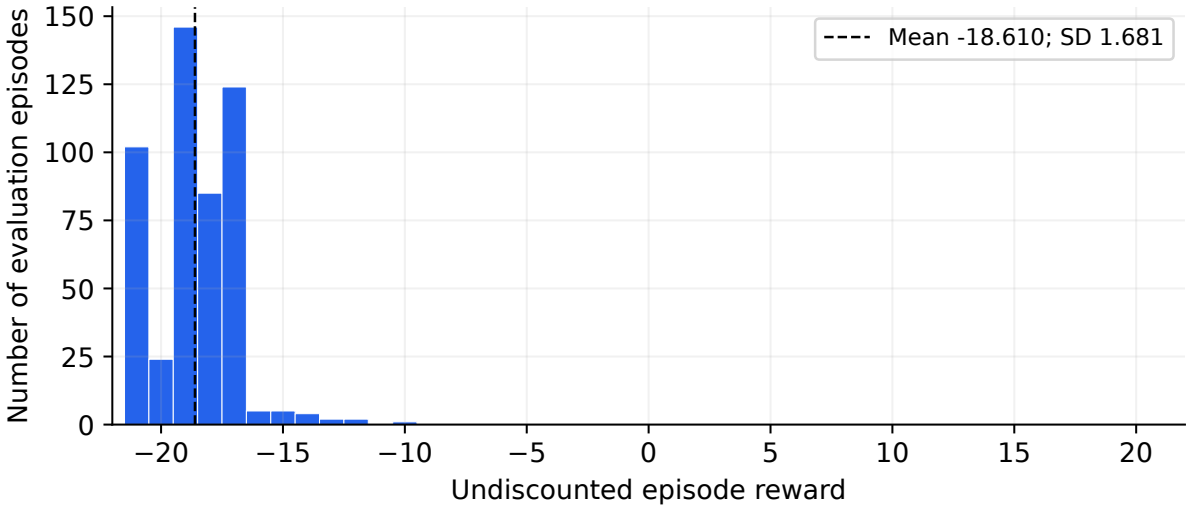


Figure 4: Pong, Part 1: all 500 final-policy evaluation rewards, with integer-score bins. Mean -18.610, population SD 1.681.

An untrained-policy reference scored -20.250 ± 0.935 over 32 games. The final policy improved modestly on that reference, but its final 100-game training average was still −18.37. The reference uses fewer episodes and different seeds, so it is a descriptive comparison rather than a formal paired significance test.

6 Part 2: CartPole with a learned baseline

The same primary seed, actor architecture, Adam settings and 8,000-episode budget were repeated with the learned baseline described in Section 2. The final model scored **260.308** ± **57.013**. Four evaluation episodes reached 500; the other 496 ended through physical failure.

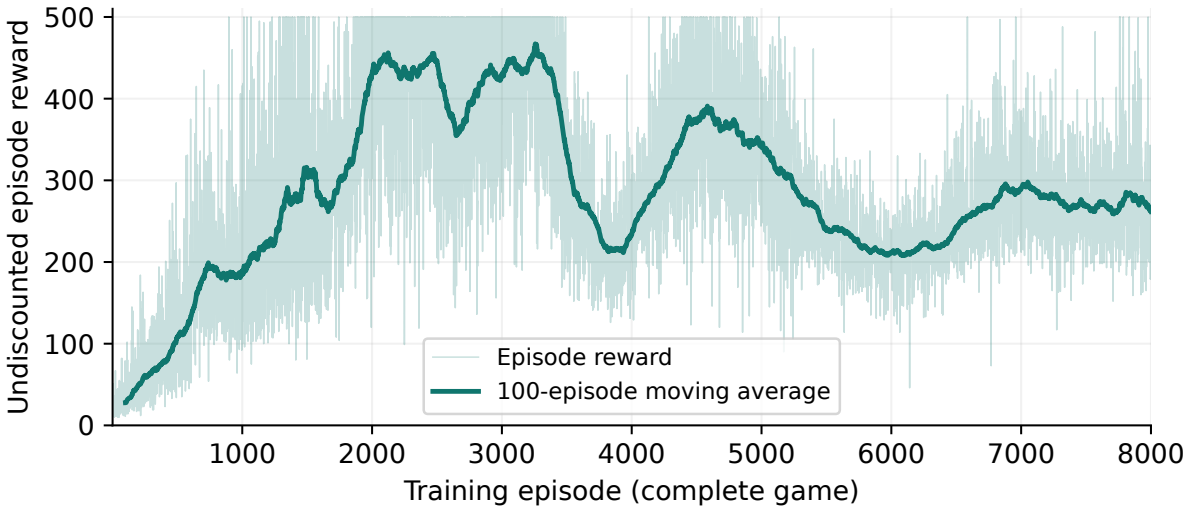


Figure 5: CartPole, Part 2: every training reward and trailing 100-episode moving average for REINFORCE with the learned state/time baseline. Seed 1668.

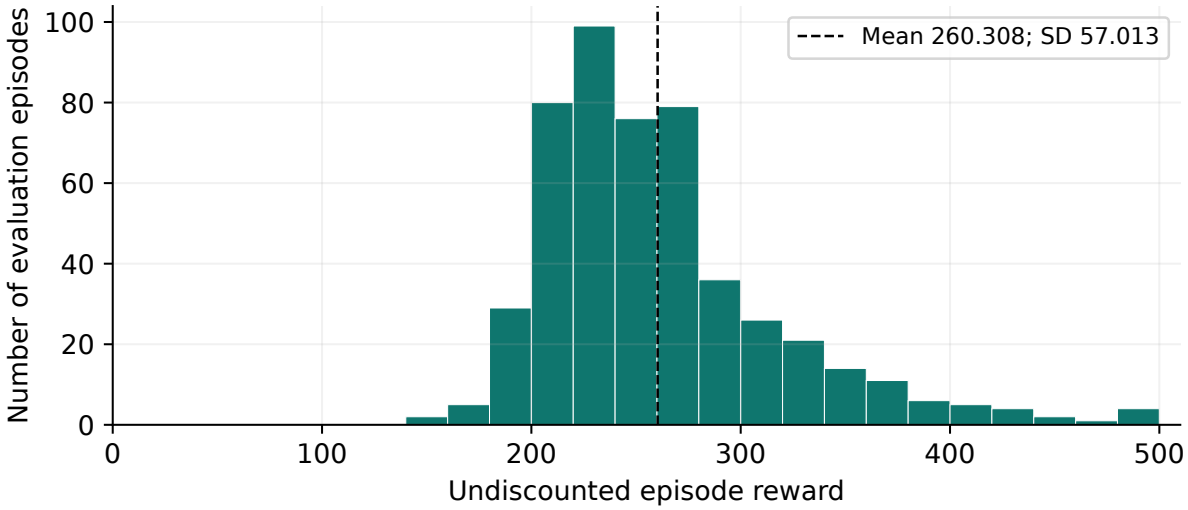


Figure 6: CartPole, Part 2: histogram of 500 final-policy evaluation rewards. Mean 260.308, population SD 57.013.

The baseline method improved markedly over random and untrained policies but performed worse than the primary basic method. It used the same number of environment episodes while doing extra computation to fit the baseline. A baseline's potential to reduce gradient variance is not a guarantee of better final control; Section 9 measures that distinction directly.

7 Part 2: Pong with a constant EMA baseline

Pong was repeated for 20,000 games with identical initial actor parameters, action restrictions and episode/reset schedules, adding the lagged constant baseline. The final policy scored **-18.446 ± 1.851**. Scores ranged from −21 to −5; all 500 games were losses and none hit the time limit.

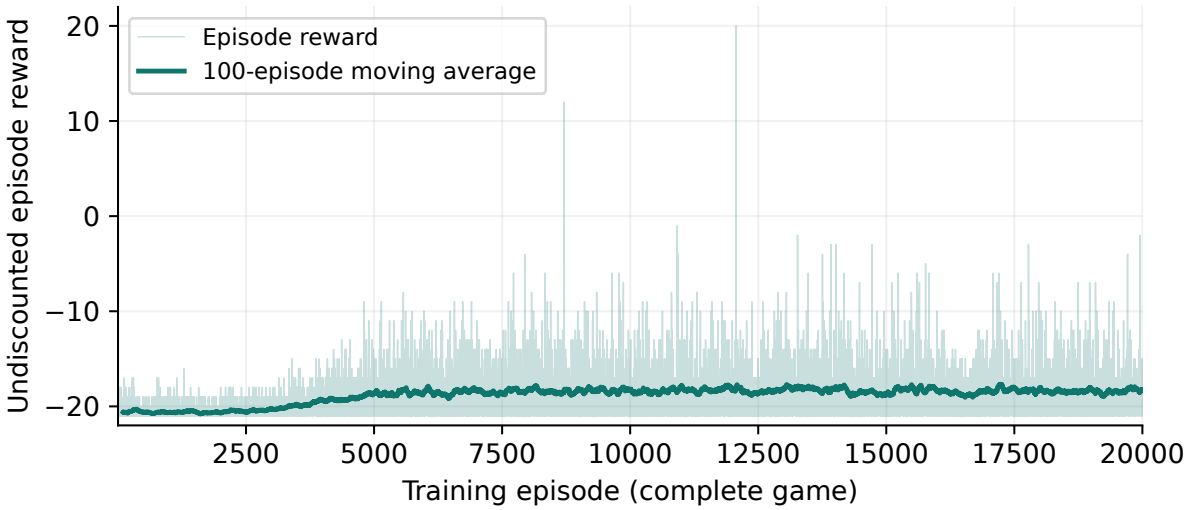


Figure 7: Pong, Part 2: full-game rewards and trailing 100-game moving average with the constant EMA baseline. Seed 1668; 20,000 games.



Figure 8: Pong, Part 2: all 500 final-policy evaluation rewards. Mean -18.446, population SD 1.851.

The mean exceeds the basic method by only 0.164 points, and the episode SD is larger. One training seed per method is insufficient to establish general superiority. The last 100 training games averaged −18.29. Later fixed-policy variance measurements also show that this EMA baseline did not consistently reduce variance (Section 11).

8 Comparison across runs and objective limitations

8.1 CartPole: retaining all training seeds

Table 3: All CartPole final checkpoints. Each row has 500 evaluation episodes. "At 500" is the fraction reaching the maximum episode score.

Method	Training seed	Mean	Population SD	At 500
No baseline	1668	500.000	0.000	1.000
No baseline	1669	271.824	28.856	0.000
No baseline	1670	483.886	65.298	0.928
Learned baseline	1668	260.308	57.013	0.008
Learned baseline	1669	360.516	79.141	0.100
Learned baseline	1670	349.940	76.321	0.092

Across the three training seeds, the average of the evaluation means was 418.570 without a baseline and 323.588 with it; sample SDs across these means were 127.341 and 55.057. These describe *between-training-run* variation, not the episode SDs in the table. The baseline outperformed the basic method for seed 1669, but not for the other two. Three seeds do not justify a broad significance claim or a claim that adding a baseline necessarily improves performance.

8.2 Pong: final versus validation-selected checkpoints

The primary figures use final policies after 20,000 games. For transparency, auxiliary checkpoint selection used only separate 32-game validations, never the 500-game test rewards:

Table 4: Supplementary Pong validation-selected policies, each separately tested for 500 games. These do not replace the final-policy results.

Method	Selected training game	Test mean	Test SD
No baseline	19,456	−18.704	1.725
EMA baseline	9,216	−18.466	1.815

Neither selected policy won a test game. The fixed-budget results therefore do not conceal an earlier checkpoint that solved the task.

8.3 Discounted return can improve while full-game score remains poor

In CartPole, +1 rewards imply discounted return $(1 - 0.95^T)/0.05$, which approaches 20. The primary baseline model's mean discounted evaluation return was 19.999743, versus approximately 20 for the basic model, despite their large difference in undiscounted scores.

In Pong, the untrained reference's mean discounted return was −1.309180, compared with 0.313913 and 0.293346 for the final basic and baseline models. Thus the recorded discounted objective improved even though full-game scores remained strongly negative. At $\gamma = 0.99$, a reward 500 decisions away has weight about 0.00657, and one 1,000 decisions away has weight about 4.32×10^{-5}. This creates a plausible credit-assignment limitation for games averaging over 1,250 decisions in evaluation. It does not by itself establish the cause of failure; a controlled ablation would be needed.

9 Correctness and edge-case experiments: CartPole

The following experiments test numerical predictions and task behavior. They complement the observed improvement over random policies; they do not use AI's agreement as evidence of correctness. Run `python validate.py` in the CartPole project to execute them.

9.1 Return calculation and gradient direction

For rewards $(1, 1, 1)$ and $\gamma = 0.95$, the independently calculated returns are $(2.8525, 1.95, 1)$. The implementation agrees to float32 precision. Setting $\gamma = 0$ returns the immediate rewards exactly. These experiments check discount exponents and the absence of reward leakage between decisions.

In an exactly enumerable two-step problem, both actions follow a Bernoulli probability $p = \text{sigmoid}(\theta)$, and each reward equals the binary action. The exact objective is $(1+\gamma)p$; its derivative is $(1+\gamma)p(1-p)$. Summing the policy-gradient estimator over all four action sequences produced 0.471188756, with or without a constant baseline, compared with analytic derivative 0.471188748. This checks the reward-to-go and outer-discount convention as well as baseline invariance.

A positive-return update for one selected action increased its probability from 0.500000 to 0.502500. An incorrect loss sign would reverse this direction.

9.2 Physical termination, truncation and reproducibility

Two runs with a fixed reset seed and the same ten actions produced identical state sequences. Separately placing the cart beyond its allowed position and the pole beyond its allowed angle caused physical termination; the terminal step still received +1. A one-decision time-limit experiment caused truncation without physical failure. The implementation ends on either flag, resets before another episode, and uses zero continuation after the specified episodic horizon. This checks end-condition logic against known environment behavior, rather than only checking array formats.

The full ten-decision trace is inspected in Section 12. It records state, action probabilities, sampled actions and rewards so individual decisions can be examined independently of a learning curve.

9.3 Does the learned baseline reduce gradient variance?

A policy was frozen at initialization. A separate baseline was fitted on 512 episodes using 150 MSE/Adam steps, then both gradient estimators were evaluated on the *same* 512 different trajectories. This is a separate variance experiment, not the evolving five-critic-steps-per-training-batch procedure. Summing sample variances over policy parameters gave:

Table 5: CartPole fixed-policy gradient variance, measured as the trace of the sample covariance matrix.

Estimator	Sum of parameter variances
No baseline	5.338262
Fitted baseline	0.842421

The reduction is 84.22%. The distance between empirical mean gradients was 0.0474, compared with a root-sum-squared standard error of 0.1176 for their paired difference. This is consistent with, but does not prove, the expected-gradient identity. It establishes a variance benefit at this frozen policy; it does not contradict the weaker final baseline training results or prove variance reduction throughout training.

10 Correctness and edge-case experiments: Pong

Pong was launched only after 15 preflight checks passed. Preflight evidence is preserved in `preflight/results.json`. The checks include the following task-specific predictions and controls.

10.1 Returns across points and action restrictions

For rewards $(0, 1, 0, -1)$ and $\gamma = 0.99$, the expected reward-to-go values are

$$(G_0, G_1, G_2, G_3) = (0.019701, 0.0199, -0.99, -1).$$

The implementation matched these within 10^{-6}. In particular, the negative reward still affects returns before the earlier positive point: returns are not accidentally reset at scoring boundaries. A separate zero-discount test returned the immediate rewards.

The two-step enumerated-gradient experiment was repeated at $\gamma = 0.99$ and agreed with its analytic derivative within 10^{-7}, with and without an action-independent baseline. A positive-return update increased the chosen action's probability; zero returns without a baseline produced zero gradient.

Complete preflight games generated scoring events and reached an episode end using only actions 2 and 3. Runtime workers reject any other agent action, and verify the environment's action names. This tests actual progress without silently introducing a FIRE or NOOP wrapper. Recorded final evaluations contained 500 normal terminations and zero truncations for each model, ruling out evaluation scores generated by an accidentally shortened game.

10.2 Visual preprocessing and stochastic replay

Synthetic images with known pixels checked the crop, factor-two sampling, removal of colors 144 and 109, preservation of foreground pixels, and lack of mutation of the original RGB frame. Real-game inputs were checked to be zero on reset and otherwise to take values only in $\{-1, 0, 1\}$ after differencing. Resetting the previous frame prevents an artificial motion signal from leaking between games.

Two emulator runs with the same reset seed and fixed actions produced identical frame/reward hashes even with v5's sticky actions. This demonstrates controlled reproducibility without removing the stochasticity required by the chosen environment version. Stochastic policy actions remain separate from emulator randomness.

10.3 Learning-rate, resource and recovery experiments

Two copies of the same initial policy received one update using identical full-game data. Mean forward KL from the old policy was 3.423×10^{-4} at learning rate 0.01 and 1.996×10^{-6} at 0.001. These quantify relative update size, not a long-run learning-rate comparison.

Preflight checked exactly one visible CUDA device and performed real GPU updates. Both policies used that one device; eight CPU workers ran the emulators. The complete suite finished in approximately 2.64 hours. There was no multi-GPU training.

An actor and its Adam state were saved after an update, restored into another copy, and both received the same next fixed-data update. The resulting parameter hashes matched exactly. This tests optimizer recovery, not an extra on-policy learning experiment. Source/config hashes, finite parameters, backup checkpoints, independent episode RNG seeds and saved evaluation progress protected the long run. Initial actors matched across methods, and every current batch used a baseline frozen from earlier data. These operational checks supplement the gradient and behavior experiments; they do not establish successful task learning.

11 Pong baseline variance over training

A baseline is intended to reduce variance in gradient estimates, not necessarily the variance of episode rewards. To isolate its effect, each probe freezes one actor and computes both estimators on the same held-out trajectories, without updating that actor or its training statistics.

At initialization, a candidate constant baseline was estimated from eight independent games and measured on 16 held-out games. The summed per-parameter gradient variance changed from 369.320 to 20.763, a 94.38% reduction. This is a *separately fitted candidate*, not the actual first training batch's EMA baseline, which starts at zero.

Table 6: All later Pong probes. Each uses 16 paired held-out games. V_0 is summed gradient variance with no baseline; V_b uses the probe's constant baseline. A ratio below one means measured reduction.

Actor's training method	Training games	V_0	V_b	V_b/V_0
EMA-baseline actor	4096	381.66	333.33	0.873
EMA-baseline actor	8192	160.81	163.52	1.017
EMA-baseline actor	12288	217.75	201.19	0.924
EMA-baseline actor	16384	118.86	120.18	1.011
Basic actor	4096	437.78	53.44	0.122
Basic actor	8192	434.45	291.59	0.671
Basic actor	12288	193.44	199.18	1.030
Basic actor	16384	200.39	224.50	1.120

For actors trained without a baseline, a hypothetical constant was fitted on eight separate games solely for the probe; it was never used in those actors' training. For actors trained with the EMA baseline, the probe used the actual frozen EMA value from prior training. These are within-actor comparisons: comparing variance levels *between different actors* would confound the baseline with changes in policy.

The actual EMA baseline's ratios were 0.873, 1.017, 0.924 and 1.011. It therefore reduced the measured variance at some checkpoints and slightly increased it at others. The separately fitted baseline also stopped consistently helping later basic-method actors. These findings do not support a claim of sustained variance reduction from this constant baseline. With only 16 trajectories per checkpoint, the estimates are noisy, and no confidence interval for the ratios was computed.

Each probe also saves the difference between empirical mean gradients and its paired standard-error measure, plus a check that actor parameters remain unchanged. The analytic expected-gradient identity is stronger than finite-sample equality. In addition, these variance probes measure the *unclipped* estimator; the frequent clipping during actual Pong training is a separate source of bias and altered update variability.

These results motivate a state-dependent baseline and more samples per variance probe as future ablations. They do not justify changing or selectively omitting the reported constant-baseline run.

12 Inspecting decisions rather than only curves

12.1 CartPole: the first ten decisions

The saved trace uses the final primary basic policy, reset seed 9100000 and action-sampling seed 123456. The four state components below are cart position x, velocity $\dot{x}$, pole angle α in radians, and angular velocity $\dot{\alpha}$. $P(1)$ is the probability of pushing right; actions 0 and 1 mean left and right. All ten rewards were 1, and neither end flag was set.

Table 7: Actual CartPole decisions, with probabilities before sampling the action.

t	x	$\dot{x}$	α	$\dot{\alpha}$	$P(1)$	Action
0	0.01276	-0.01188	-0.04659	-0.03077	0.1019	0
1	0.01252	-0.20630	-0.04721	0.24686	0.8335	1
2	0.00840	-0.01054	-0.04227	-0.06033	0.0700	0
3	0.00818	-0.20503	-0.04348	0.21872	0.7711	0
4	0.00408	-0.39951	-0.03910	0.49738	0.9930	1
5	-0.00391	-0.20385	-0.02915	0.19263	0.7348	1
6	-0.00798	-0.00833	-0.02530	-0.10911	0.0428	0
7	-0.00815	-0.20308	-0.02748	0.17549	0.6828	1
8	-0.01221	-0.00757	-0.02397	-0.12574	0.0337	0
9	-0.01236	-0.20234	-0.02649	0.15929	0.6266	0

At decision 3, left was sampled although right had probability 0.7711. This is valid stochastic behavior: probability is not a deterministic instruction. The changing angular velocity also shows why inspecting the pole angle alone would not explain every action.

12.2 Pong: the beginning, a scored point, and the ending

The final basic policy's first evaluation game has a compressed image-difference trace and CSV. The first ten decisions, the first point's neighborhood, and the end are saved; skipped middle decisions are not presented as consecutive steps.

Table 8: Selected actual Pong decisions. $P(2)$ is the probability of ALE action 2; G_t includes all subsequent point rewards, not only the next point.

t	$P(2)$	ALE action	Reward	G_t	γ^t
0	0.2772	2	0	0.59704	1
1	0.0007	3	0	0.60307	0.99
2	0.0064	3	0	0.60917	0.98
3	0.0003	3	0	0.61532	0.97
4	0.9862	2	0	0.62154	0.961
5	0.0000	3	0	0.62781	0.951
6	0.1081	3	0	0.63415	0.941
7	1.0000	2	0	0.64056	0.932
8	0.6196	2	0	0.64703	0.923
9	0.0000	3	0	0.65357	0.914
93	0.8358	2	1	1.52031	0.393
1797	0.0007	3	-1	-1.00000	1.43e-08

Step 0 has zero motion input. The point at step 93 is positive, whereas the late loss at 1797 has weight only 1.43×10^{-8}. This illustrates discounting, not its causal effect on learning. The `*_frames.npz` files retain the corresponding 80×80 motion inputs.

13 How to reproduce and validate the results

The repository is organized into `mlds_assignment1_cartpole`, `mlds_assignment1_pong`, and `mlds_assignment1_policy_gradient_report`. All algorithm, validation, plotting and report-generation source is included. Preserve the recorded outputs before rerunning commands that write into their run directories.

13.1 CartPole: train, evaluate and run validation

In a Python environment with PyTorch, install the CartPole requirements and run:

```
cd mlds_assignment1_cartpole
python -m pip install -r requirements.txt
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
jupyter lab cartpole_plots.ipynb
```

The suite trains all six models and evaluates each for 500 episodes. The validation script executes the experiments in Section 9, raises an error on a failed assertion, and writes `validation/validation.json`. The notebook regenerates plots without training. To evaluate one saved model:

```
python cartpole.py evaluate \
  --checkpoint runs/no_baseline_seed1668/last.pt
```

13.2 Pong: preflight and a fresh single-GPU reproduction

Activate `~/mlds_assignment1_pong_env` on deepdish5. On a new host, first create that virtual environment and install the Pong `requirements.txt`. For a fresh reproduction, copy `pong_env.py`, `pong_core.py`, `run_pong.py`, `plot_results.py`, `supervise.sh` and `config.json` into a new directory, without old checkpoints or preflight outputs. Then run:

```
export CUDA_VISIBLE_DEVICES=GPU-cef65388-88b0-f39a-6bac-c5316eb78540
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

The UUID is the original assigned card; on another host, update both the config and supervisor to one available GPU before preflight. Preflight runs Section 10's checks; the job adds variance probes every 4,096 games. Existing checkpoints trigger resume, not new training.

Inspect `preflight/results.json` for controls and versions, `runs/*/training.csv` for training, `runs/*/evaluation/test_final.json` for individual test rewards, and `diagnostics/` for variance and trace data. The source README documents checkpoint recovery and all output paths.

13.3 Rebuild this report without training

Run `python build_assets.py` in this report folder beside the two saved experiment folders, then compile `POLICY_GRADIENT_REPORT.tex` with Tectonic or pdfLaTeX twice. The script independently recomputes statistics and all eight plots. Repository metadata comes from `submission.json`; detailed build instructions are in the README.

14 What would be tested with three additional months?

14.1 CartPole: distinguish reliable learning from a favorable seed

Run substantially more seeds with separate hyperparameter-validation and final-test resets. Sweep Adam learning rate and batch size under matched episode budgets. Compare the current learned state/time baseline with a constant baseline and different critic capacities or fitting frequencies. Measure critic error and paired gradient variance at early, middle and late actor checkpoints, not only at initialization. This would test whether baseline approximation or update lag contributes to the weaker primary run.

Stress-test the final policies using controlled initial pole angles, cart velocities and changed pole/cart physics. Evaluate stochastic and greedy policies separately. These experiments would determine whether the perfect primary evaluation generalizes beyond the standard small reset region and whether the observed cross-seed variability persists.

14.2 Pong: investigate failure with controlled ablations

First, retain the required v5 environment and actions 2/3, but compare frame differencing with two- or four-frame stacks and a small convolutional policy. This tests whether the present features discard useful position information. Track point outcomes, first-point performance and complete-game score separately, without replacing full games with point episodes in the main comparison.

Second, examine credit assignment: measure how much of the unclipped gradient comes from successive time windows, and compare discounting conventions only in explicitly labeled sensitivity experiments. Keep the required $\gamma = 0.99$ result as the main experiment. For example, restarting returns at points would change the objective and must not be presented as the original full-game algorithm.

Third, tune learning rate, gradient cap and batch size using held-out validation seeds. Compare clipping frequencies and action saturation as well as returns. Repeat with several training seeds, and reserve untouched seeds for final 500-game evaluations. Longer training alone would not distinguish a poor representation, optimization problems and a weak late-game reward signal.

14.3 Baselines: test the mechanism, not only the score

For Pong, compare the current lagged scalar with learned state-dependent and time-dependent baselines. Fit and test each on independent trajectories at the same frozen actors. Use hundreds of paired trajectories per probe, bootstrap uncertainty for variance ratios, and compare clipped and unclipped gradients. This would establish when a baseline helps, whether apparent small increases are sampling noise, and whether reduced variance translates into better task performance.

References

[1] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 229–256 (1992). https://doi.org/10.1007/BF00992696.

[2] Farama Foundation. Cart Pole environment documentation. https://gymnasium.farama.org/environments/classic_control/cart_pole/.

[3] Farama Foundation. Pong environment documentation. https://ale.farama.org/environments/pong/.